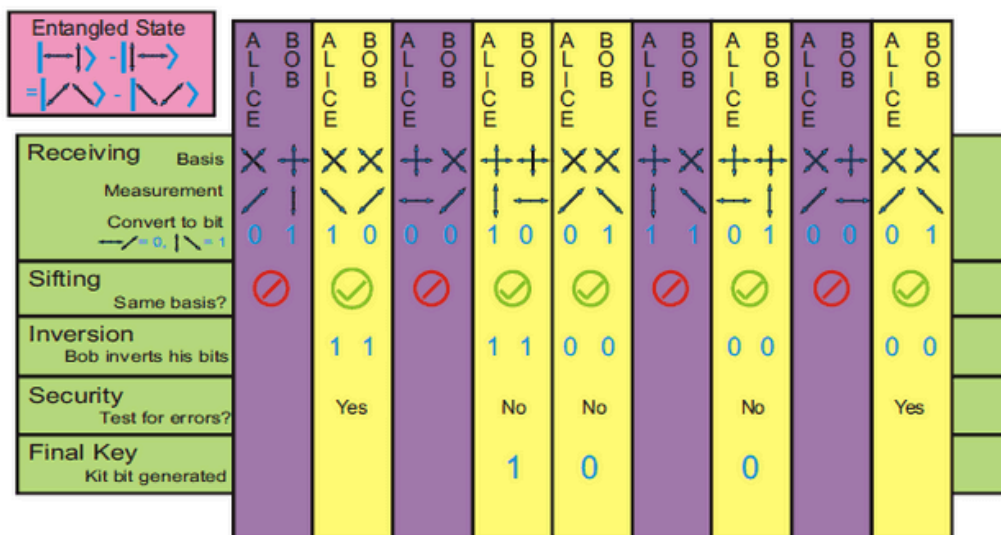


BBM92 Quantum Key Distribution

The BBM92 protocol [1], named after its inventors Charles Bennett, Gilles Brassard and David Mermin in 1992, is an entanglement-based Quantum Key Distribution protocol. This challenge is intended for people to familiarize with the SquidASM set-up in a practical environment, and we assume no background on quantum physics. For this, we will guide the team on a practical implementation [2,3] of the original BBM92 protocol.

The main goal of the protocol is for two parties, Alice and Bob, who share entangled qubits (EPR pairs) and an authenticated classical channel, to agree on the same classical bitstring (symmetric key), while ensuring that a third-party eavesdropping on their classical communication (1) does not learn any bits of the key they agreed on, and (2) any attempt of the third-party on tampering with the outcome will be detected.

BBM92 leverages *entanglement* to establish beyond-classical correlations between two parties (which will provide a symmetric key), and *measurement uncertainty*, that is, according to quantum mechanics there are complementary measurements for which you cannot possibly get certain outcomes simultaneously when measuring a qubit. Do not worry if this sounds unfamiliar! We will guide you through the protocol and you will see what it means in practice.



The above figure depicts a schematic description of BBM92 for 9 entangled qubit pairs, in short there is one round of quantum communication and measuring (denoted receiving in the figure) where the two parties will obtain correlated bitstrings, and there are 3 rounds of classical post-processing (sifting, security and final key, SquidASM's choice of entanglement makes an inversion phase unnecessary). If you implement all the steps you will be left with Alice and Bob sharing a secret symmetric key!

The protocol goes as follows:

1. Receiving (set-up phase): there are 2 types sockets in SquidASM, a classical one `csocket` and a quantum one `epr_socket`; we will start with quantum communication where Alice makes a request for half of a maximally entangled photon pair (the easiest example of this in SquidASM are EPRs) and send the other half to Bob (who must also do a request). Alice can make her requests in the quantum socket using `create_keep()[0]`, and Bob needs to be listening to such a request using `recv_keep()[0]`. Actually `create_keep(number=n)` generates a list of EPRs where by default $n = 1$, hence we need to select our local EPR using `[0]`. It is then required for both parties to send the instructions to the Quantum Network Processing Unit using `yield from connection.flush()`.
2. Receiving (measurement phase): Afterwards, Alice and Bob must choose randomly, and independently, which bases they want to measure on. As mentioned, in quantum mechanics measurements in *complementary bases* of the same qubit cannot give certain outcomes, or in other words, once a measurement is done, it cannot be undone (hence given one qubit you cannot obtain information about two complementary measurements). The easiest complementary bases to implement on SquidASM are the *Computational (Z)* which is done directly on qubit `q` with the command `q.measure()` and the *Hadamard (X)* which can be implemented by first applying a Hadamard operation to the qubit `q.H()` followed by a computational measurement. After this step Alice and Bob should each have a list of measurement bases and a list of outcomes.
3. Sifting: in this step Alice and Bob will check if their independent measurements were done in the same bases, since they have a useful correlation only if they did so. For that, they will tell each other which bases they chose (note that this step can be public, as they are only sharing the bases choice and not the measurement outcome, an adversary that does not have access to the measurement outcome does not learn anything from the bases choice). You can send classical messages to peers in SquidASM using classical sockets with `csocket.send(message)`, but in order to receive the message the other party must be listening with `yield from csocket.recv()`. After sharing their bases, both parties should go through the list and identify the instances where they chose the same basis, whilst discarding the mismatched bases rounds entirely. At the end of this step the measurement outcomes associated with the same basis choices form what is called the *raw key*, and in presence of no noise nor adversaries these should already be equivalent for both parties.
4. Security: however, it might be the case that there was indeed either noise or an adversary in the communication channel, thus a security check is needed. This is typically done by estimating the Quantum Bit Error Rate (QBER): Alice and Bob sacrifice random parts of the raw key (say half of it) and compare them publicly, the QBER is the percentage of the compared bits that was faulty. A low QBER (say 10%) can be assigned to noisy channels, but an active adversary could induce a significant error percentage (as high as 25%). At the end of this step Alice and Bob must give an estimate of the QBER.
5. Final key: you probably got a 0% QBER in the above step because we have not induced any error nor an adversary, but nevertheless since we publicly announced parts of the raw key we must discard them, the remaining elements of the raw key form what is called the *final key* -- check that it is indeed the same for both Alice and Bob!

Doing a security analysis without noise or an adversary is not very interesting, but we invite you to see what happens if you add some either in the quantum channel (in this case the fidelity $\in [0, 1]$) or the

errors in the measurement devices: you can find the set-up configuration in `config.yaml`. Moreover, different runs of the protocol will provide different outcomes; do you feel confident to provide some statistics? That is, high is the QBER and how long is the final key on average given different number of EPR pairs?

If you are comfortable with the challenge, you can also analyze what happens if you have an active eavesdropper by adding a third node.

1. You can start by considering an honest intermediate node (say Eve) that shares an EPR pair with Alice and another one with Bob. Eve can perform an entanglement SWAP so that logically Alice and Bob share an EPR by making a Bell state measurement between her two qubits: this is done with a 2-qubit CNOT gate `qubit_1.cnot(qubit_2)` followed by a Hadamard gate in qubit 1, and measuring both in the Computational basis. Eve will have to communicate the classical measurement outcomes to Bob, who can do local corrections (if the outcome of the first measurement is apply a `Z()` gate, if the outcome of the second measurement is apply an `X()` gate).
2. You can also consider an adversarial intermediate node, who lies about the correction operations, what effect does this have on the QBER and the size of the final key?

[1] <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.68.557>

[2] <https://arxiv.org/pdf/2505.24851v1>

[3] [https://www.aliroquantum.com/hubfs/White Papers/Aliro White Paper 2025-06-19 Quantum Keys in Action_ Using BBM92 for Secure Communication Today.pdf](https://www.aliroquantum.com/hubfs/White%20Papers/Aliro%20White%20Paper%202025-06-19%20Quantum%20Keys%20in%20Action_Using%20BBM92%20for%20Secure%20Communication%20Today.pdf)